

```

#include <iostream>
#include <cstdlib>
#include <ctime>

#define SIZE 100

using namespace std;

class Node
{
public:
    Node()
    {
        next = NULL;
        pre = NULL;
    }
    Node(int n)
    {
        data = n;
        next = NULL;
        pre = NULL;
    }
    int getData() { return data; }
    Node *getNext() { return next; }
    Node *getPre() { return pre; }
    void setData(int d) { data = d; }
    void setNext(Node *n) { next = n; }
    void setPre(Node *p) { pre = p; }

private:
    int data;
    Node *next, *pre;
};

class List
{
public:
    List() { list = NULL; }
    List(int n) { generate(n); }

    void generate(int n)
    {
        int j;
        list = NULL;
        for (j = 0; j < n; j++)
            generate();
    }

    void generate()
    {
        Node *buf = new Node(rand() % 100); // Limited range for easier debugging
        buf->setNext(list);
        if (list != NULL)
            list->setPre(buf);
        list = buf;
    }

    void bubbleSort()
    {
        if (!list) return; // If the list is empty, return.
    }

```

```

bool swapped;
Node *current;
Node *last = NULL;

do
{
    swapped = false;
    current = list;

    while (current->getNext() != last)
    {
        if (current->getData() > current->getNext()->getData())
        {
            // Swap the two nodes
            Node *nextNode = current->getNext();

            // Adjust pointers to swap nodes
            Node *pre = current->getPre();
            Node *nextNext = nextNode->getNext();

            if (pre) pre->setNext(nextNode);
            nextNode->setPre(pre);

            nextNode->setNext(current);
            current->setPre(nextNode);

            current->setNext(nextNext);
            if (nextNext) nextNext->setPre(current);

            if (current == list) list = nextNode; // Update the head if needed

            swapped = true;
        }
        else
        {
            current = current->getNext();
        }
    }
    last = current;
} while (swapped);
}

void selectionSort()
{
    if (!list) return; // If the list is empty, return.

    Node *start = list;

    while (start != NULL)
    {
        Node *min = start;
        Node *current = start->getNext();

        while (current != NULL)
        {
            if (current->getData() < min->getData())
            {
                min = current;
            }
        }
    }
}

```

```

        current = current->getNext();
    }

    if (min != start)
    {
        // Swap start and min
        Node *preMin = min->getPre();
        Node *nextMin = min->getNext();
        Node *preStart = start->getPre();
        Node *nextStart = start->getNext();

        if (preMin) preMin->setNext(start);
        start->setPre(preMin);

        start->setNext(nextMin);
        if (nextMin) nextMin->setPre(start);

        if (preStart) preStart->setNext(min);
        min->setPre(preStart);

        min->setNext(nextStart);
        if (nextStart) nextStart->setPre(min);

        if (start == list) list = min; // Update the head if needed
    }

    start = start->getNext();
}

void insertionSort()
{
    if (!list) return; // If the list is empty, return.

    Node *sortedEnd = list;
    Node *current = list->getNext();

    while (current != NULL)
    {
        Node *nextCurrent = current->getNext(); // Save the next node to be processed

        Node *sorted = sortedEnd;

        // Find the correct position to insert the current node in the sorted part
        while (sorted != NULL && sorted->getData() > current->getData())
        {
            sorted = sorted->getPre();
        }

        if (sorted == sortedEnd)
        {
            sortedEnd = current; // Move the sortedEnd pointer forward
        }
        else
        {
            // Remove the current node from its place
            Node *preCurrent = current->getPre();
            Node *nextCurrent = current->getNext();

            if (preCurrent) preCurrent->setNext(nextCurrent);

```

```

        if (nextCurrent) nextCurrent->setPre(preCurrent);

        // Insert it after sorted
        if (sorted == NULL)
        {
            current->setNext(list);
            current->setPre(NULL);
            list->setPre(current);
            list = current;
        }
        else
        {
            Node *nextSorted = sorted->getNext();
            sorted->setNext(current);
            current->setPre(sorted);
            current->setNext(nextSorted);
            if (nextSorted) nextSorted->setPre(current);
        }
    }

    current = nextCurrent; // Move to the next node
}

```

```

void print()
{
    Node *cur = list;
    while (cur != NULL)
    {
        cout << cur->getData() << " ";
        cur = cur->getNext();
    }
    cout << endl;
}

```

```

private:
    Node *list;
};

```

```

int main()
{
    srand(time(NULL));

    cout << "Bubble Sort:" << endl;
    List *l = new List(10);
    l->print();
    l->bubbleSort();
    l->print();

    cout << "\nInsertion Sort:" << endl;
    l = new List(10);
    l->print();
    l->insertionSort();
    l->print();

    cout << "\nSelection Sort:" << endl;
    l = new List(10);
    l->print();
    l->selectionSort();
    l->print();
}

```

}